



CCS3300-G3 OPERATING SYSTEMS

Group Presentation

Beyond the Desktop: Mobile & Embedded OS

Architecture, Challenges, and Convergence

YUE CHENGHAO	227154
WANG KAILUN	227046
HUA JIE	226758



Source: www.geeksforgeeks.org

What is a Non-Conventional OS?

The Concept

Conventional OS (Windows, macOS) assumes:

- › Unlimited power source.
- › Standard Inputs (Mouse/Keyboard).
- › General purpose tasks (Do anything reasonably well).

Non-Conventional OS flips the script:

It is an OS designed for **specific contexts** where standard assumptions fail.

Key Differentiators



Resource Constrained

Limited Battery, RAM, Thermal limits.



Specialized I/O

Touch, Sensors, Voice, GPIO pins.



Timing Critical

Often requires real-time responses (RTOS).

The Core Conflict



Desktop OS

Unlimited Power: Plugged into the wall.

Precision Input: Mouse &
Keyboard.

High TDP: Performance is the priority.



Mobile OS

Battery Constraints: Every cycle counts.

Touch Input: Fat finger, lower precision.

Connectivity: Unstable networks.

"Constraints define Architecture"

A Tale of Two Philosophies

> Android (Google)

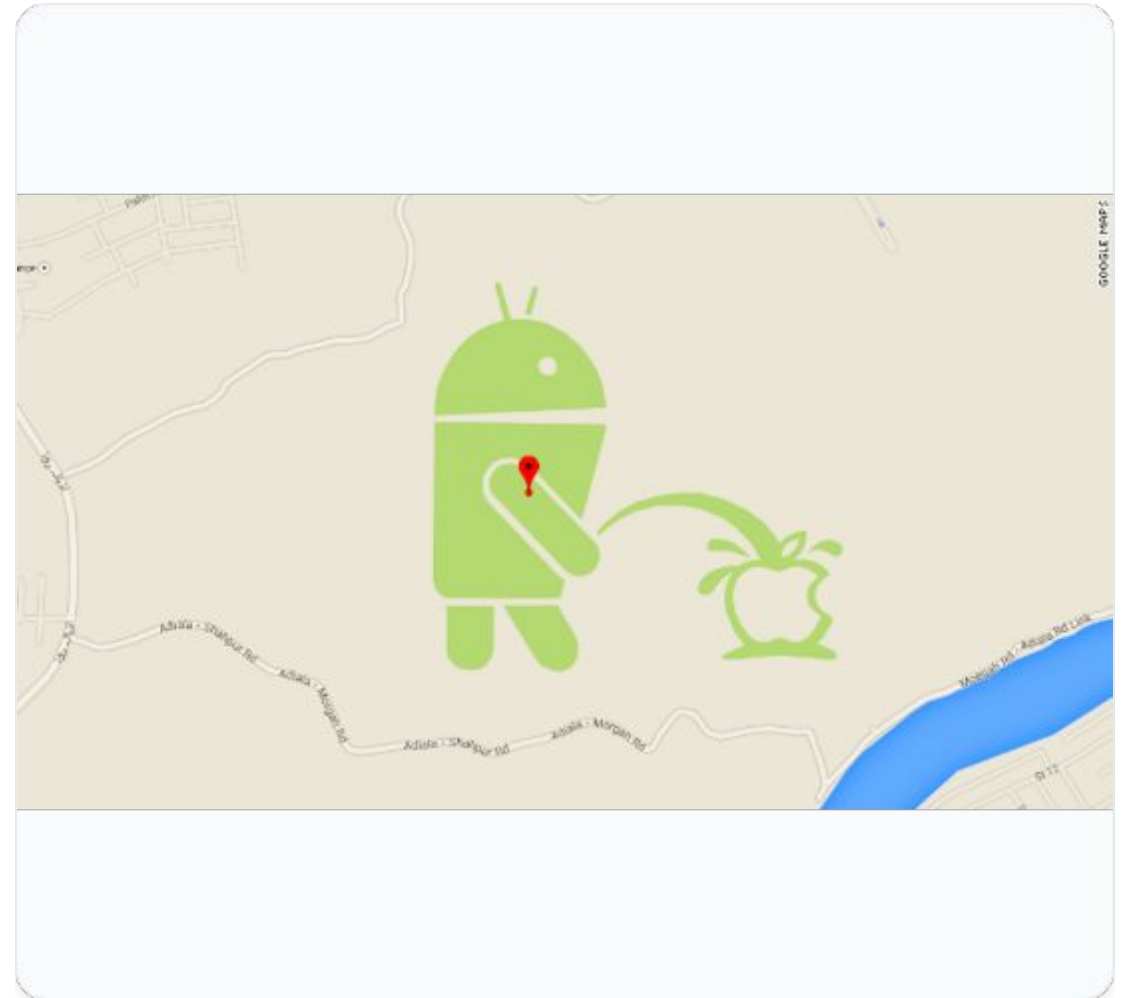
Based on Modified Linux Kernel.

Philosophy: *Open, Diverse, Universal.*

> iOS (Apple)

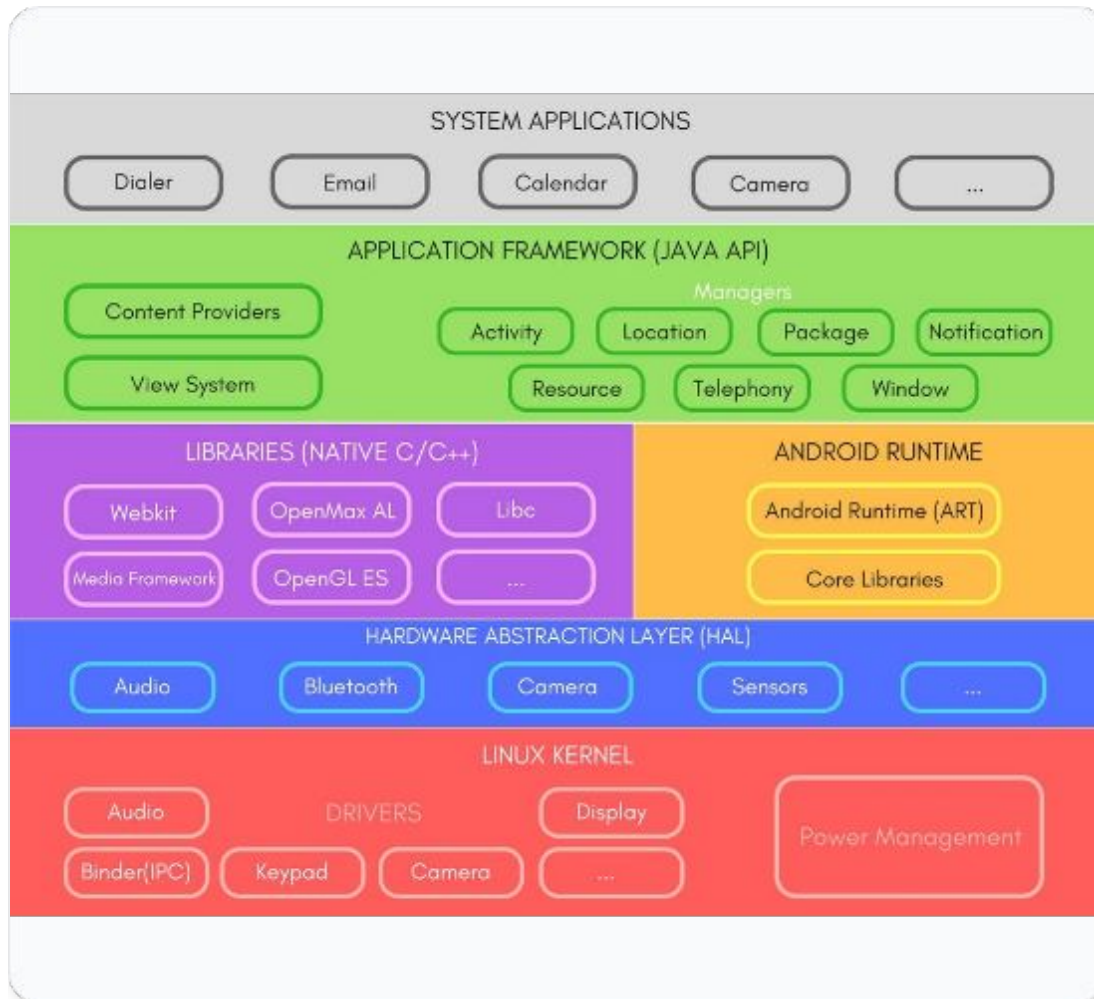
Based on XNU Hybrid Kernel (Mach + BSD).

Philosophy: *Walled Garden, Integrated, Optimized.*



Source: www.creativebloq.com

Android: The Universal Translator



Hardware Abstraction Layer (HAL)

The secret sauce that allows Android to run on thousands of different chips (Snapdragon, MediaTek, etc.).

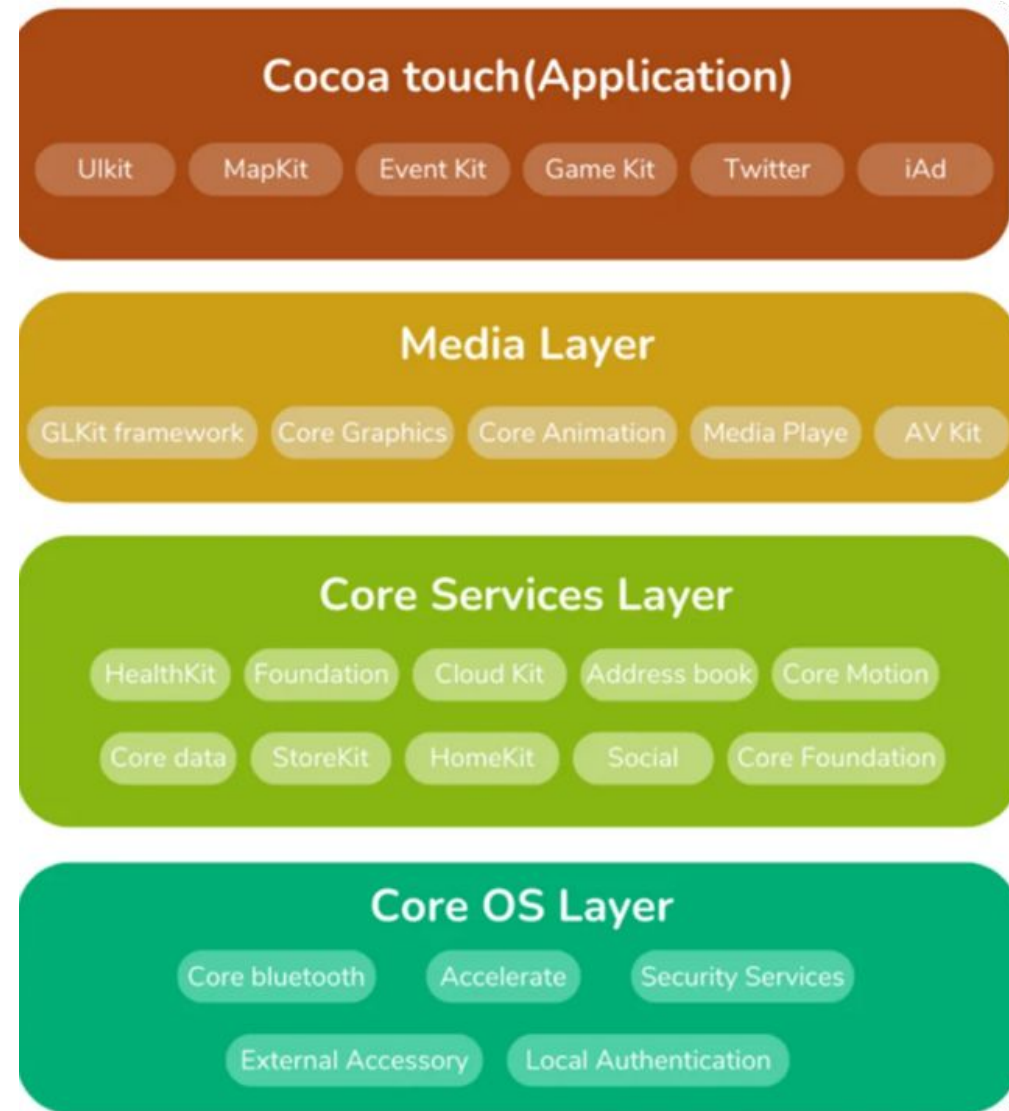
- > **Problem:** Hardware vendors want to keep drivers proprietary.
- > **Solution:** HAL provides a standard interface, decoupling implementation from the framework.
- > **Communication:** Uses **Binder IPC**, optimized for speed and low overhead compared to standard Linux pipes.

iOS: The Speed Demon

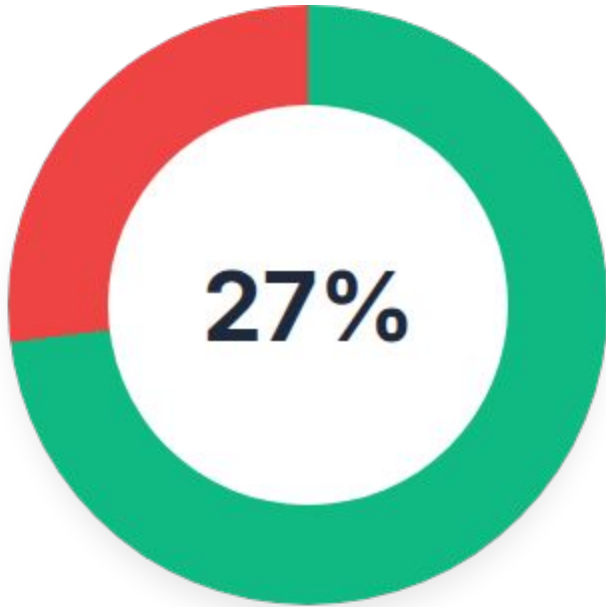
Optimization First

Apple controls both hardware and software, eliminating the need for a heavy abstraction layer.

- > **UI Priority:** The Touch interface gets highest processing priority. Background tasks are paused to ensure 60/120 FPS smoothness.
- > **Sandboxing:** Every app runs in a strictly isolated container (sandbox).
- > **Security:** Apps cannot directly access other apps' data, enhancing privacy and system stability.



The Price of Freedom: Fragmentation



Only 27% of users on Android 14
(Majority on older versions)

A Developer's Nightmare

While iOS users update almost instantly, Android users are scattered across years of OS versions.

- > **Challenge:** Testing apps across hundreds of screen sizes and OS APIs.
- > **Solution: Project Treble.** Separates the core Android OS from vendor implementation, making updates faster and easier.

Embedded OS: The Invisible Scalpel

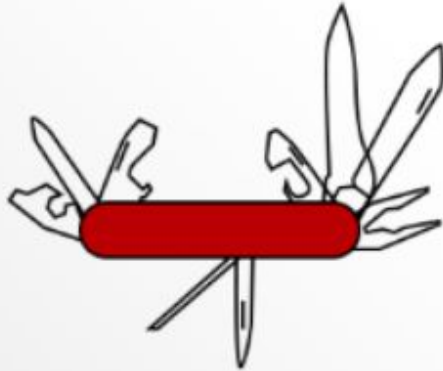
General vs. Dedicated

If Mobile OS is a **Swiss Army Knife** (does everything okay), Embedded OS is a **Scalpel** (does one thing perfectly).



Scalpel

Easy to market/explain
Easy to adopt
Does one thing well
Quick to build & test



Swiss Army Knife

Hard to market/explain
Hard to adopt
Does nothing well
Long time to build

Mission: Run 24/7 for 10 years without crashing.

Key Trait: Determinism & Reliability.

Examples: Washing machines, ABS Brakes, Pacemakers.

GPOS



Linux



iOS



Android

RTOS



VxWorks



QNX



INTEGRITY

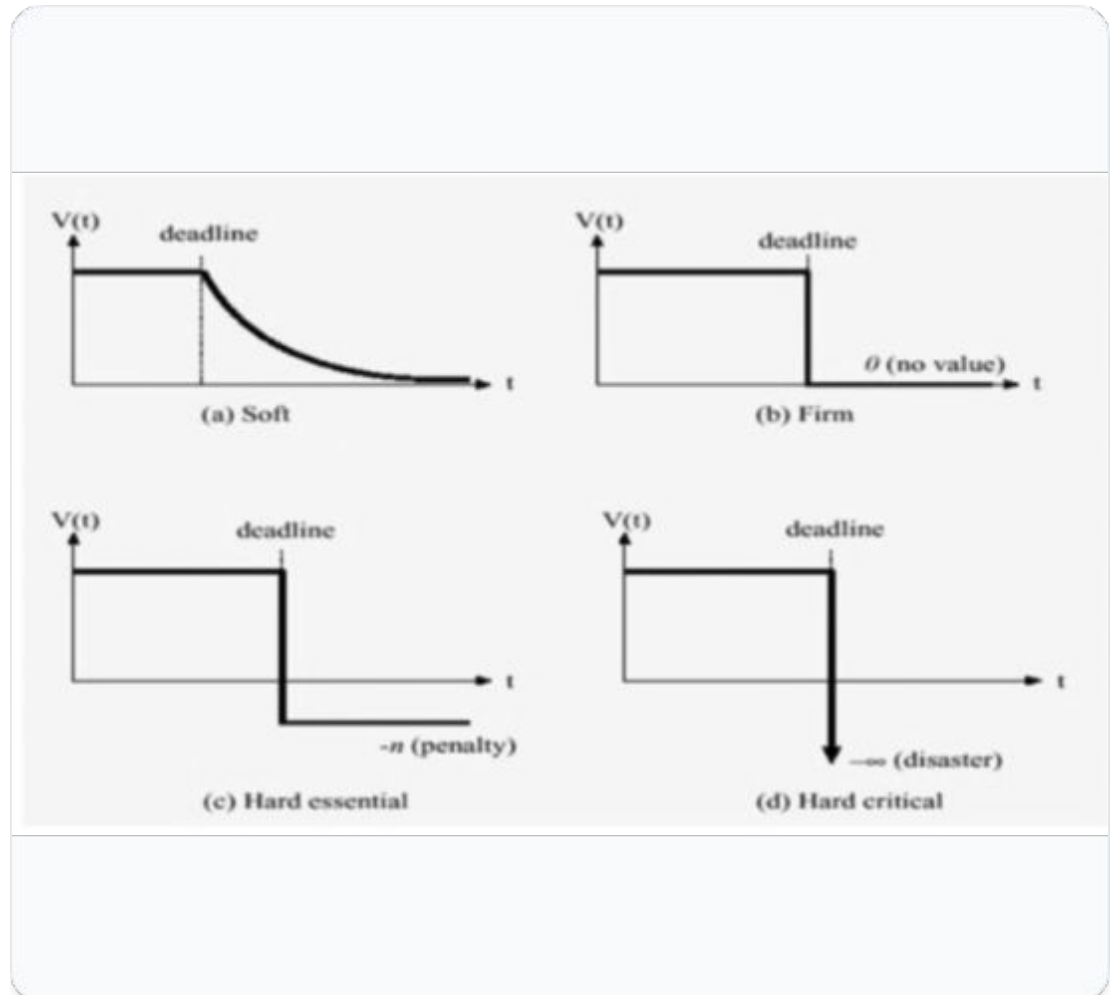
RTOS: Real-Time Operating System

What is "Real-Time"?

It doesn't mean "Fast". It means **Deterministic**.

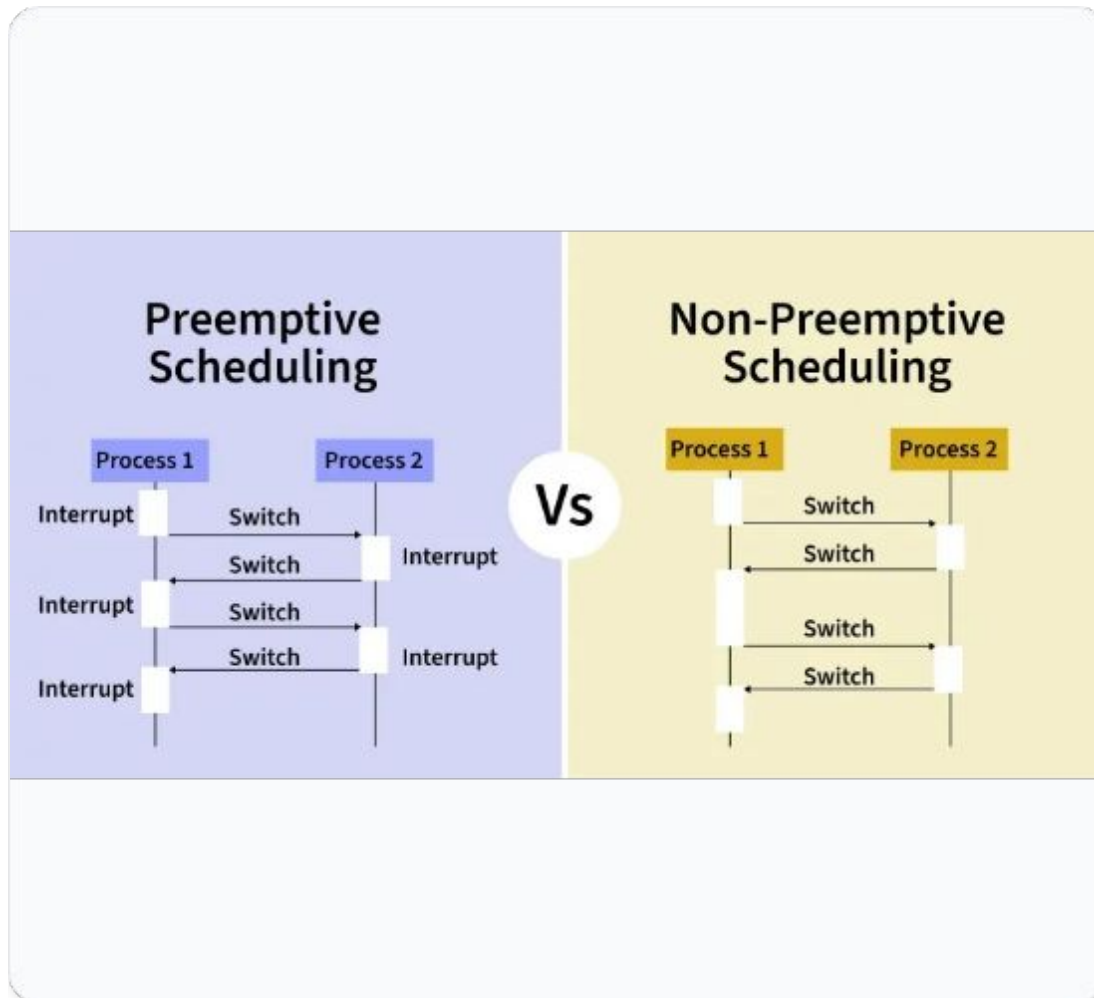
- **General OS (Windows/Android):** "I'll deploy the airbag when I'm done updating this app." (**Best Effort**)
- **RTOS:** "I will deploy the airbag in exactly 0.001 seconds, no matter what." (**Hard Deadline**)

Hence, RTOS is essential for safety-critical embedded systems, not general-purpose computing.



Source: redfoxsec.com

Under the Hood: Preemptive Scheduling



No Fairness, Only Priority

In a General OS (Linux CFS), the scheduler tries to be fair to all apps. In RTOS, fairness does not exist.

- > **Preemption:** If a High Priority task (e.g., Brakes) wakes up, the OS *immediately* pauses the Low Priority task (e.g., Radio).
- > **Latency:** Context switch happens in microseconds ().
 μs

Living on a Budget



Extreme Constraints

Kilobytes of RAM (not
Gigabytes).
MHz CPU speeds.



No Malloc()

Dynamic memory allocation is
banned to prevent
fragmentation. Everything is
statically allocated.



Tickless Idle

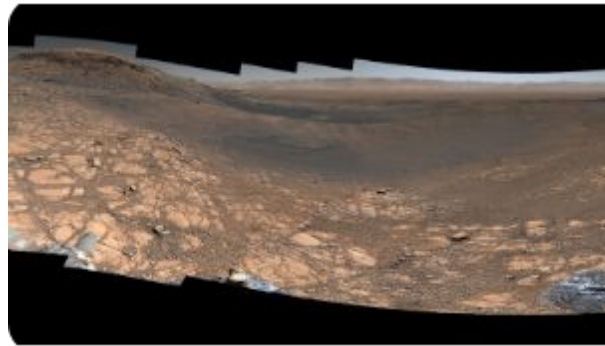
CPU sleeps deeply between
interrupts to run on a coin
battery for years.

Industry Standards



FreeRTOS

Runs on billions of IoT devices.
Small footprint, highly portable.



Source: www.jpl.nasa.gov

VxWorks

The OS of Mars Rovers.
Extreme reliability for critical
aerospace missions.



Embedded Linux

For complex gateways requiring
networking and richer UI (e.g.,
Smart Home Hubs).

Convergence: Computer on Wheels

Software Defined Vehicle

Modern EVs (Tesla, Android Automotive) create a massive conflict of requirements.

- **Entertainment:** Users want Spotify, Maps, and slick animations (Requires Android/Linux).
- **Safety:** Brakes and Speedometer must NEVER fail (Requires RTOS).
- **The Risk:** You cannot run brakes on Android. If the app crashes, the car cannot stop.



The Solution: Virtualization

The Hypervisor

A virtualization layer that runs directly on the hardware, allowing two OSs to run simultaneously.

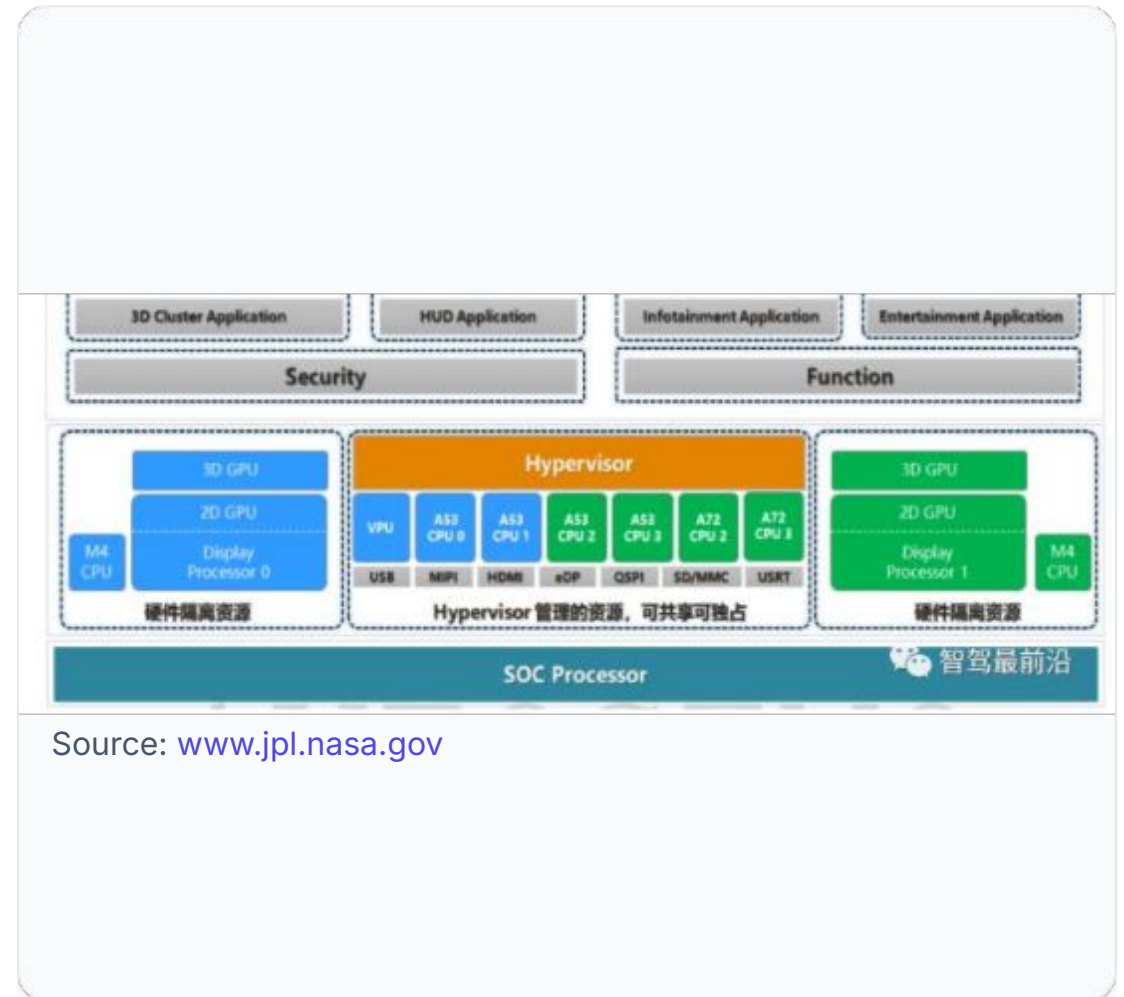
Guest 1: Android

Runs Music & Maps.
Can crash safely.

Guest 2: QNX (RTOS)

Runs Cluster & Brakes.
Never stops.

Isolation: Even if the infotainment screen freezes, the speedometer keeps working.



Source: www.jpl.nasa.gov

The Frontiers

TinyML



Shrinking AI models to fit on microcontrollers. Devices can now "think" locally without Cloud latency.

Matter



The unified language for IoT. Finally connects Apple, Google, and Amazon devices into one secure mesh network.

Conclusion: The Spectrum



Mobile OS

Focus on **Human Experience**
and Ecosystem.
Flexibility & Features.



The Trade-off

The future Architect must know
when to demand Flexibility vs.
Reliability.



Embedded OS

Focus on **Machine Control** and
Determinism.
Safety & Speed.

References



Mobile Operating Systems (Android & iOS)

- [1]Android Open Source Project. (n.d.). *Architecture overview*. Android Source. Retrieved January 9, 2026, from <https://source.android.com/docs/core/architecture>
- [2]Esper. (n.d.). *How Project Mainline is solving Android's fragmentation problem*. Esper Blog. Retrieved January 9, 2026, from <https://www.esper.io/blog/what-is-project-mainline>
- [3]Ivanisevic, M. (n.d.). *What the Zygote!? Android OS boot process explained*. Medium. Retrieved January 9, 2026, from <https://medium.com/@voodoomio/what-the-zygote-76f852d887d9>

Embedded Systems & RTOS

- [4]Amazon Web Services. (n.d.). *Low power support - FreeRTOS*. AWS Documentation. Retrieved January 9, 2026, from <https://docs.aws.amazon.com/freertos/latest/userguide/low-power-support.html>
- [5]ByteSnap Design. (n.d.). *FreeRTOS vs Linux for embedded systems*. ByteSnap. Retrieved January 9, 2026, from <https://www.bytesnap.com/news-blog/freertos-vs-linux-embedded-systems/>
- [6]GeeksforGeeks. (2026). *What's difference between MMU and MPU?* Retrieved January 9, 2026, from <https://www.geeksforgeeks.org/operating-systems/whats-difference-between-mmu-and-mpu/>
- [7]Oreate AI. (n.d.). *Comprehensive analysis of FreeRTOS task scheduling and priority management*. Oreate AI Blog. Retrieved January 9, 2026, from <https://www.oreateai.com/blog/comprehensive-analysis-of-freertos-task-scheduling-and-priority-management-from-tick-to-delay/29d677497a212a9fb8f5553e3b083715>

Convergence & Industry Applications (Automotive, IoT, Medical)

- [8]Android Automotive. (n.d.). *QNX vs AAOS (Android Automotive OS): Compare the platforms*. Retrieved January 9, 2026, from <https://androidautomotive.dev/blog/qnx-vs-aaos>
- [9]BlackBerry. (2024, August). *In surgical robotics, buying time saves lives*. BlackBerry Blog. Retrieved January 9, 2026, from <https://blogs.blackberry.com/en/2024/08/in-robotics--buying-time-saves-lives>
- [10]Qorvo. (n.d.). *Simplifying smart homes: Learn how Matter, Thread and Wi-Fi are revolutionizing IoT connectivity*. Qorvo Design Hub. Retrieved January 9, 2026, from <https://www.qorvo.com/design-hub/blog/simplifying-smart-homes-learn-how-matter-thread-and-wi-fi-are-revolutionizing-iot-connectivity>